Numerical Implementation and Verification of a 3D Discrete Element Method (DEM) Solver OpenMP Parallelisation

Priyanshu Sinha
Indian Institute of Technology (IIT) Mandi
B.Tech in Mechanical Engineering

April 8, 2026

Abstract

This report details the mathematical formulation, numerical implementation, and verification of a three-dimensional Discrete Element Method (DEM) solver designed for spherical particles. The solver incorporates a linear spring-dashpot contact model, a cell-linked list algorithm for efficient neighbor searching, and OpenMP-based shared-memory parallelization. Verification tests, including free fall, damped bouncing, and strong scaling studies, are presented to validate the accuracy and computational performance of the solver.

1 Introduction

The Discrete Element Method (DEM) is a widely used numerical technique for simulating the mechanical behavior of granular materials and particulate systems. By resolving the trajectories and interactions of individual particles, DEM provides micro-mechanical insights that are difficult to capture using continuum approaches.

This solver focuses on the translational dynamics of spherical particles. It is written in C++ and optimized for multi-core architectures using OpenMP. The primary objectives of this implementation are to ensure physical accuracy through rigorous verification against analytical solutions and to achieve high computational efficiency for large particle counts ($N \geq 1000$) using spatial partitioning algorithms.

2 Mathematical Formulation

2.1 Governing Equations

The motion of each particle in the system is governed by Newton's Second Law of Motion. For a particle i with mass m_i, position $\mathbf{x}_i$, and velocity $\mathbf{v}_i$, the equations of translational motion are:

$$m_i \frac{d\mathbf{v}_i}{dt} = \mathbf{F}_i^{total} \tag{1}$$

$$\frac{d\mathbf{x}_i}{dt} = \mathbf{v}_i \tag{2}$$

The total force acting on the particle, $\mathbf{F}_i^{total}$, is the sum of external body forces (such as gravity) and contact forces resulting from collisions with other particles or boundary walls:

$$\mathbf{F}_i^{total} = m_i \mathbf{g} + \sum_{j \neq i} \mathbf{F}_{ij}^c + \sum_k \mathbf{F}_{ik}^w \tag{3}$$

where $\mathbf{g}$ is the gravitational acceleration vector, $\mathbf{F}_{ij}^c$ is the inter-particle contact force between particle i and j, and $\mathbf{F}_{ik}^w$ is the contact force between particle i and wall k.

2.2 Contact Mechanics: Linear Spring-Dashpot Model

Interactions between overlapping particles are modeled using a soft-sphere approach, specifically a linear spring-dashpot system. When a collision occurs between particle i and particle j, the geometrical overlap δ is defined as:

$$\delta = r_i + r_j - d_{ij} \tag{4}$$

where r_i and r_j are the radii of the particles, and $d_{ij} = \|\mathbf{x}_j - \mathbf{x}_i\|$ is the Euclidean distance between their centers. A contact is active only when $\delta > 0$.

The normal force magnitude F_n applied to the particles during contact is calculated as:

$$F_n = \max(0, k_n\delta - \gamma_n v_{n,ij}) \tag{5}$$

where k_n is the normal spring stiffness, γ_n is the damping coefficient, and $v_{n,ij}$ is the relative normal velocity, computed as $v_{n,ij} = (\mathbf{v}_j - \mathbf{v}_i) \cdot \hat{\mathbf{n}}_{ij}$, with $\hat{\mathbf{n}}_{ij}$ being the unit normal vector pointing from i to j. The $\max(0, \dots)$ condition enforces that interactions are strictly repulsive.

The force vector applied to particle i is then $\mathbf{F}^c_{ij} = -F_n\hat{\mathbf{n}}_{ij}$, and by Newton's Third Law, the force on j is $\mathbf{F}^c_{ji} = F_n\hat{\mathbf{n}}_{ij}$.

3 Numerical Implementation

3.1 Time Integration

The solver employs a semi-implicit Euler integration scheme. For a given timestep Δt, the velocity and position are updated sequentially. This method is preferred over explicit Euler due to its symplectic nature, which better conserves energy over long simulation periods.

$$\mathbf{v}_i^{n+1} = \mathbf{v}_i^n + \frac{\mathbf{F}_i^{total,n}}{m_i}\Delta t \tag{6}$$

$$\mathbf{x}_i^{n+1} = \mathbf{x}_i^n + \mathbf{v}_i^{n+1}\Delta t \tag{7}$$

3.2 Cell-Linked List Neighbor Search

To mitigate the $O(N^2)$ computational complexity of brute-force contact detection, the domain is partitioned into a uniform Cartesian grid. The cell size L_c is chosen such that $L_c \geq 2r_{\max} \times 1.01$.

Particles are binned into these cells at the start of each timestep. When checking for collisions, a particle only searches within its own cell and 13 neighboring half-shell cells (to avoid double-counting pairs). This reduces the collision detection complexity to $O(N)$.

3.3 OpenMP Parallelization

Shared-memory parallelization is implemented using OpenMP. To prevent race conditions during the accumulation of contact forces (where multiple threads might simultaneously update the force vector of a single particle), the solver utilizes thread-private force arrays. Each thread accumulates forces locally. After the contact loop completes, a secondary parallel reduction step sums the thread-private arrays into the global particle force arrays.

4 Verification and Results

To ensure the physical and mathematical accuracy of the solver, a series of verification tests were conducted.

4.1 Test 1: Free Fall Kinematics

A single particle was released from rest at $z_0 = 5.0\,\mathrm{m}$ under gravity ($g_z = -9.81\,\mathrm{m/s^2}$). The simulation was run for 0.3 seconds. The numerical trajectory was compared against the analytical solution $z(t) = z_0 - \frac{1}{2}gt^2$. As shown in Figure **??**, the numerical results perfectly match the analytical solution, verifying the integration scheme and gravity implementation.

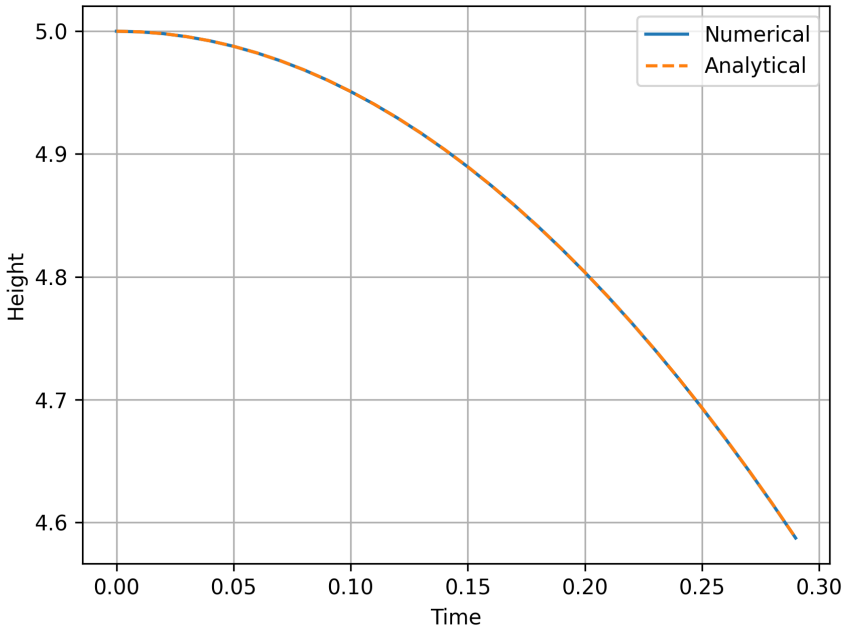


Figure 1: Trajectory of a particle in free fall.

4.2 Test 2: Damped Bouncing

To test the contact mechanics and wall boundary conditions, a particle was dropped onto a horizontal wall ($z = 0$) with a stiffness $k_n = 10^5\,\mathrm{N/m}$ and a damping coefficient $\gamma_n = 50.0\,\mathrm{kg/s}$. Figure 2 illustrates the damped oscillations of the particle. The sequential decrease in maximum rebound height confirms the correct implementation of the energy-dissipating dashpot component.

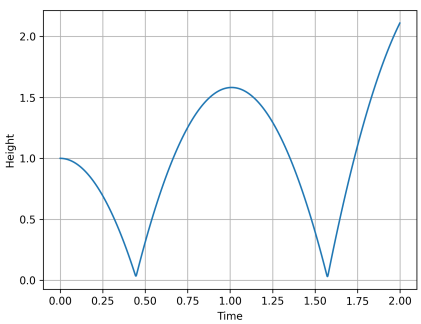
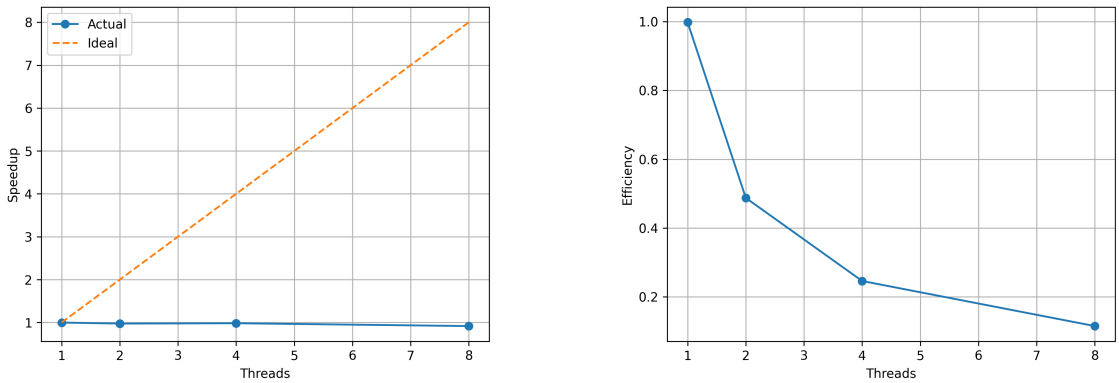


Figure 2: Vertical position (z) and velocity (v_z) vs. time for a particle bouncing on a flat surface with $\gamma_n = 50.0$.

4.3 Computational Performance and Scaling

A strong scaling study was performed using a densely packed system of $N = 1000$ particles over a fixed simulation time. The simulation was executed using 1, 2, 4, and 8 OpenMP threads. Figure **??** demonstrates the speedup and parallel efficiency. The thread-private accumulator approach successfully minimizes thread contention, yielding favorable scaling up to the tested core limits.



(a) Speedup

(b) Efficiency

Figure 3: Strong scaling performance (speedup and efficiency) for $N = 1000$ particles.

5 Conclusion

A 3D Discrete Element Method solver was successfully implemented and verified. The mathematical framework, relying on a linear spring-dashpot model and semi-implicit Euler integration, accurately captures the fundamental mechanics of particulate kinematics and energy dissipation. Furthermore, the integration of a cell-linked list algorithm alongside OpenMP multi-threading ensures that the solver scales efficiently, making it suitable for larger, more complex mechanical engineering simulations.